



ENTREGA INTERMEDIA

Visualización y Análisis de Datos – Entrega Intermedia

Sergio Gil López

1. INTRODUCCIÓN

Objetivos

El objetivo de esta entrega intermedia es garantizar que los datos adquiridos en la fase de extracción se han transformado con éxito en un *dataset* tabular estructurado, limpio y de alta calidad, listo para su ingesta en herramientas de visualización interactiva.

Objetivo Global

Analizar la oferta de alojamiento vacacional en diferentes mercados turísticos de España (Toledo, San Sebastián, Valencia y Málaga) mediante técnicas avanzadas de visualización de datos, con el fin de identificar patrones de precios dinámicos basados en la antelación de reserva (*days_ahead*), la ocupación (*guests*), la ubicación geoespacial y las características estructurales de los inmuebles.

Objetivos Secundarios

Mapear la oferta de alojamientos, utilizando las coordenadas interceptadas del tráfico de red, para visualizar la saturación espacial y evaluar la correlación entre la prima de precio y la distancia kilométrica al centro histórico.

Comparar la variación del precio en función de la antelación temporal de la reserva y la estacionalidad entre destinos de turismo interior (Toledo), costa norte (San Sebastián), levante (Valencia) y sur (Málaga).

Analizar la información desestructurada extraída de las descripciones o títulos de los anuncios para identificar características de valor añadido y visualizar su impacto directo en el precio por noche.

Fuentes de Datos

Para la consecución de los objetivos, se usará una arquitectura de datos propia:

Web Scraping Avanzado: Se utilizará un conjunto de datos propio, actualmente en fase de recolección, generado mediante un *script* en Python que utiliza la librería `Playwright`. Este sistema realiza un barrido automatizado de plataformas de alquiler vacacional y consta de dos vías de extracción:

Extracción DOM: Captura de variables estructuradas directamente de la interfaz web (precio, camas, dormitorios).

Intercepción de Red: Captura de respuestas JSON asíncronas para extraer las coordenadas GPS exactas de cada inmueble, los identificadores únicos y fragmentos de texto desestructurado (títulos y descripciones).

A día de la redacción de esta entrega intermedia se tienen 9.707 entradas. Sin embargo, se están obteniendo un mínimo de 2000 entradas al día, por lo que para la fecha de entrega final se espera tener más de 20.000 entradas, lo cual, se considera suficiente para el alcance del trabajo.

2. DATOS ORIGINALES

Acceso al conjunto de datos original

Dadas las características volumétricas de los datos extraídos y siguiendo las directrices de la asignatura para conjuntos de datos pesados, se han alojado los archivos originales en un repositorio en drive para garantizar su disponibilidad:

- Dataset extraído ("results.csv"):
 - <https://drive.google.com/file/d/1Xqt0DTHF4L7IV4Q7UoFgZQvs0X7ibJCq/view?usp=sharing>
 - Descripción: Archivo .csv con los datos extraídos de Airbnb compuesto actualmente por 9.707 registros pertenecientes a Toledo, Valencia, San Sebastián y Málaga.

Descripción de datos originales

El proceso de recolección genera un conjunto de datos tabular estructurado cuyas filas representan observaciones únicas (la oferta de un alojamiento para una fecha y unas condiciones concretas). Al tratarse de un conjunto de datos complejo y heterogéneo, la tabla original incluye tanto variables numéricas y categóricas como campos de texto libre (no estructurado).

A continuación, se detalla el diccionario de variables correspondiente al conjunto de datos original:

Variables de Control de Simulación (Inputs de Búsqueda)

- *"scrape_date"* (Fecha): Fecha exacta en la que el bot realizó la extracción de la información (formato AAAA-MM-DD). Permite controlar el factor temporal de la recolección.
- *"city"* (Texto): Ciudad objetivo sobre la cual se ejecutó la iteración de búsqueda (ej. "Toledo", "Valencia").
- *"checkin_date"* (Fecha): Fecha simulada de entrada al alojamiento turístico.
- *"days_ahead"* (Numérico): Días de antelación con los que se simula la reserva respecto a la fecha actual (ej. 14, 30, 60 días). Fundamental para el análisis dinámico de precios.
- *"guests"* (Numérico): Número de huéspedes estipulados en la búsqueda (ej. 2 o 4 personas).

Variables Estructuradas del Alojamiento (DOM)

- *"id"* (Numérico/Texto): Código identificador único del alojamiento dentro de la plataforma. Fundamental para evitar duplicados reales y realizar el seguimiento temporal del mismo inmueble.
- *"price"* (Numérico): Precio total por noche ofrecido en la plataforma, expresado en euros (€).
- *"beds"* (Numérico): Cantidad de camas anunciadas por el anfitrión.
- *"bedrooms"* (Numérico): Cantidad de dormitorios o estancias independientes ofertadas.

Variables Geoespaciales y Desestructuradas (Intercepción de Red)

- *"dist_centro_km"* (Numérico): Distancia euclidiana o geodésica calculada matemáticamente en kilómetros. Se obtiene cruzando las coordenadas (latitud y

longitud) interceptadas en el tráfico de red del inmueble con las coordenadas absolutas del centro histórico del municipio objetivo. En este caso, este proceso lo ejecuta el script de extracción de datos, pero en realidad se podría considerar una operación de transformación de datos.

- “*Raw_text*” (Texto libre): Campo que alberga la información extraída en bruto del frontend del anuncio. Es un bloque de datos completamente desestructurado que recoge el título del anuncio, incluyendo las características más importantes de donde se extraerán la lista de servicios y características cualitativas ("superanfitrión", "alojamiento nuevo").

3. PROCESO DE LIMPIEZA Y TRANSFORMACIÓN

El conjunto de datos original, al provenir de técnicas de Web Scraping directo sobre el frontend y tráfico de red de una plataforma comercial, presentaba un alto grado de "ruido" y desestructuración. Para asegurar la fiabilidad de las futuras visualizaciones en Altair, se ha diseñado e implementado un script de procesamiento automatizado utilizando Python y sus librerías Pandas (para la manipulación del Dataframe) y Re (para el procesamiento mediante Expresiones Regulares).

El proceso de transformación se ha dividido en las siguientes fases secuenciales:

1. Corrección de codificación:

En primer lugar, se soluciona el problema de codificación de caracteres (Mojibake) derivado de la extracción en bruto. Mediante un diccionario de reemplazo, se normalizaron los caracteres especiales del idioma español (tales como "Ã" por "á", "Ñ" por "ñ", o "â" por "€") presentes en las descripciones de los anuncios, unificando además todo el texto en minúsculas para facilitar las fases posteriores.

```
# ----- 1. CODIFICACIÓN -----
correcciones = {
    'Ã': 'á', 'Ã@': 'é', 'Ã': 'í', 'Ã³': 'ó', 'Ãº': 'ú',
    'Ã±': 'ñ', 'Ã': 'Á', 'â': '€', 'Ã': ' ', ' ': ' '
}
for mal, bien in correcciones.items():
    texto = texto.replace(mal, bien)

texto = texto.lower()
```

2. Limpieza de ruido del texto libre:

La variable original que contiene el texto de los anuncios (“*raw_text*”) incluye una gran cantidad de información redundante ya capturada en otras columnas estructuradas. Por lo tanto, se aplican múltiples patrones de búsqueda (Regex) para eliminar sistemáticamente:

- Bloques de fechas residuales introducidas por la plataforma.
- Cadenas de valoraciones, puntuaciones y número de reviews (ej. "valoración media de 4.88 sobre 5").
- Precios incrustados en el texto y etiquetas promocionales (ej. "de los mejores anuncios", "cancelación gratuita").

- Repeticiones exactas de características estructurales generadas por el DOM de la web (ej. transformando "1 dormitorio 1 dormitorio" a "1 dormitorio").
- Cadenas numéricas residuales y saltos de línea.

Este proceso de da como resultado una nueva variable denominada *“texto_limpio”*, dejando exclusivamente la descripción semántica del alojamiento.

```
# ----- 2. LIMPIEZA DE RUIDO -----
patrones_a_eliminar = [
    r'del \d{1,2} al \d{1,2} [a-z]{3} \d{1,2} [a-z]{3}', # Borra fechas
    r'valoración media de.*?\d+', # Borra bloque de valoraciones
    r'\d+.*evaluaciones\d+\d+.*estrellas\d+.*valoración.*?(?:experiencia|anfitrión)?',
    r'de los mejores anuncios',
    r'recomendación del viajero',
    r'anfitrión (particular|profesional)',
    r'consulta el desglose del precio',
    r'cancelación gratuita',
    r'\d+.*€.*?en total',
    r'\d+.*€',
    r'[ , ]',
    r'[\n\r]+',
    # Borra camas y dormitorios porque YA los tienes en las columnas numéricas originales
    r'\d+.*dormitorios?',
    r'\d+.*(camas?|literas?|sofás?|s.*camas?) (?:s+(dobles?|individuales?|«queen»|king))?',
    r'\d+.*\d+.*' # Colas numéricas residuales
]

for patron in patrones_a_eliminar:
    texto = re.sub(patron, ' ', texto)

# Elimina repeticiones exactas de palabras útiles para que no ensucian pero sigan ahí
texto = re.sub(r'superanfitrión superanfitrión', 'superanfitrión', texto)
texto = re.sub(r'alojamiento nuevo nuevo', 'alojamiento nuevo', texto)

# Quita espacios múltiples
texto = re.sub(r'\s+', ' ', texto).strip()
return texto
```

3. Orden de datos:

Para facilitar la comprensión y visualización de los datos en modo tabla, se ordenan las entradas de la siguiente manera:

1. Ciudad (*“city”*)
2. Alojamiento (*“id”*)
3. Checkin Date (*“checkin_date”*)

```
# ----- 3. ORDEN -----
if "checkin_date" in df.columns and "city" in df.columns and "id" in df.columns:
    df = df.sort_values(by=["city", "id", "checkin_date"])
```

4. Corrección de errores de extracción

Se ha comprobado que el cálculo de distancias ha fallado en algunas filas, dejando sin rellenar el campo *“dist_centro_km”*. Sin embargo, al tener otras entradas del mismo alojamiento (mismo *“id”*) con la distancia calculada, se ha incluido un punto en el script para completar los huecos vacíos con la información disponible siempre que sea posible.

```
# ----- 4. CORRECCIÓN VALORES VACÍOS -----
print("🔍 Rellenando distancias al centro faltantes...")
if 'dist_centro_km' in df.columns and 'id' in df.columns:
    # Agrupa por ciudad e ID y copia el valor numérico (ignorando vacíos) a todas las
    # filas del mismo alojamiento
    df['dist_centro_km'] = df.groupby(['city', 'id'])['dist_centro_km'].transform('max')
```

5. Extracción de Características:

A partir de los datos base y del “*texto_limpio*”, se procede a la derivación de nuevas variables fundamentales para responder a los objetivos del proyecto:

A. Variables Espaciales:

Como se ha comentado anteriormente, esta parte (creación de la variable “*dist_centro_km*” mediante la fórmula de Haversine) se ejecuta durante la propia fase de extracción (Web Scraping), optimizando así el pipeline de datos.

B. Variables Categóricas:

Utilizando técnicas básicas de minería de textos sobre la variable “*texto_limpio*”, se genera un conjunto de variables booleanas. Estas columnas permiten segmentar el precio en base al valor añadido del inmueble. Las características extraídas incluyen:

- Servicios e instalaciones: “*tiene_wifi*”, “*tiene_aire_acond*”, “*tiene_piscina*”, “*tiene_parking*”, “*tiene_terraza_balcon*”, “*tiene_patio*”, “*tiene_vistas*”.
- Categorización del alojamiento: “*es_casa_rural*”, “*es_hotel*”.
- Atributos comerciales: “*es_nuevo*” (inmuebles recién incorporados al mercado), “*es_superanfitrión*” (anfitriones con máxima calificación).
- Tipología de alquiler: “*es_solo_habitacion*”, variable crítica para discernir entre el alquiler de un inmueble completo frente al alquiler parcial (habitaciones privadas o compartidas), evitando así distorsiones en el cálculo del precio medio.

```
# ----- 5. EXTRACCIÓN DE CARACTERÍSTICAS -----
# Detecta la columna con el texto largo
col_texto = 'texto_completo' if 'texto_completo' in df.columns else 'raw_text'

# Crea de una columna limpia para revisión
df['texto_limpio'] = df[col_texto].apply(limpiar_texto)

print("🔍 Extrayendo características...")
# Crea columnas booleanas (1 o 0)
df['tiene_wifi'] = df['texto_limpio'].str.contains('wifi', na=False).astype(int)
df['tiene_aire_acond'] = df['texto_limpio'].str.contains('aire acondicionado', na=False).astype(int)
df['tiene_piscina'] = df['texto_limpio'].str.contains('piscina', na=False).astype(int)
df['tiene_parking'] = df['texto_limpio'].str.contains('aparcamiento|parking|garaje', na=False, regex=True).astype(int)
df['tiene_terraza_balcon'] = df['texto_limpio'].str.contains('terraza|balcón|balcon', na=False, regex=True).astype(int)
df['tiene_patio'] = df['texto_limpio'].str.contains('patio', na=False).astype(int)
df['tiene_vistas'] = df['texto_limpio'].str.contains('vistas', na=False).astype(int)
df['es_superanfitrión'] = df['texto_limpio'].str.contains('superanfitrión', na=False).astype(int)
df['es_nuevo'] = df['texto_limpio'].str.contains('alojamiento nuevo', na=False).astype(int)
df['es_casa_rural'] = df['texto_limpio'].str.contains('casa rural', na=False).astype(int)
df['es_hotel'] = df['texto_limpio'].str.contains('hotel', na=False).astype(int)
df['es_solo_habitacion'] = df['texto_limpio'].str.contains('habitaci[ón]n', na=False, regex=True).astype(int)
```

4. RESULTADO

Tras la ejecución del pipeline de procesamiento y limpieza detallado en la sección anterior, el conjunto de datos original se transforma en un formato tabular estructurado, analítico y libre de ruido.

El dataset resultante, exportado en formato nativo de Excel (.xlsx), se estructura como una matriz de datos óptima para su posterior ingesta en la librería de visualización Altair.

A continuación, se detalla la estructura final de las variables de valor añadido que conforman el nuevo conjunto de datos:

Nuevas Variables Analíticas Generadas:

- `es_solo_habitacion` (Booleano): Identifica si el alquiler es parcial.
- `es_superanfitrion` (Booleano): Identificador de reputación del anfitrión.
- `es_nuevo` (Booleano): Identificador de alojamiento de reciente creación.
- `es_casa_rural` / `es_hotel` (Booleano): Categorización específica del inmueble.
- `tiene_wifi`, `tiene_aire_acond`, `tiene_piscina`, `tiene_parking`, `tiene_terraza_balcon`, `tiene_patio`, `tiene_vistas` (Booleanos): Variables descriptivas de los servicios de valor añadido extraídas del texto.

Acceso al conjunto de datos preparado

Con el fin de facilitar la evaluación de esta entrega intermedia y permitir la trazabilidad del proceso de limpieza, el dataset final estructurado y preparado para la fase de visualización se ha alojado en el repositorio en la nube del proyecto.

- Dataset Preparado:
 - https://docs.google.com/spreadsheets/d/1MIGFHCi2wHm7QcdRwPog6U_ICKCz3A_0/edit?usp=sharing&oid=103378895492990503087&rtpof=true&sd=true

Descripción: Archivo en formato .xlsx que contiene los registros limpios y las nuevas características extraídas, listo para la programación gráfica en Python (Altair).

5. PROPUESTA DE VISUALIZACIÓN

Una vez garantizada la viabilidad técnica del conjunto de datos mediante el proceso de extracción y estructuración detallado en las secciones previas, el siguiente y último paso del proyecto consistirá en el diseño e implementación del cuadro de mandos visual.

De acuerdo con los requisitos de la asignatura, todo el entorno gráfico se desarrollará íntegramente utilizando la librería declarativa Altair en Python. Para responder a los objetivos planteados en la introducción, se propone el siguiente esquema preliminar de visualizaciones, las cuales integrarán elementos de interactividad (filtrado cruzado, tooltips y selectores dinámicos) para aportar valor analítico (Nota: Es únicamente una propuesta, que podrá variar a la hora de realizar la parte final del proyecto):

A. Análisis Temporal: Impacto de la Antelación en el Precio

- Tipo de gráfico: Gráfico de líneas múltiple (Multi-line chart) o Mapa de Calor (Heatmap).
- Variables implicadas: *“checkin_date”* (Eje X temporal), *“price”* (Eje Y / Gradiente de color), *days_ahead* (Categoría de color / Eje Y en heatmap).

- Objetivo: Visualizar de forma clara si la curva de precios sufre variaciones significativas al simular la reserva con 14, 30 o 60 días de antelación para las mismas fechas de check-in, permitiendo identificar patrones de "ofertas de última hora" o encarecimiento por escasez.

B. Análisis Espacial: Dispersión del Precio respecto al Centro Histórico

- Tipo de gráfico: Gráfico de dispersión interactivo (Interactive Scatter Plot).
- Variables implicadas: "*dist_centro_km*" (Eje X), "*Price*" (Eje Y).
- Interactividad: El tamaño de la burbuja representará la capacidad (guests), y se aplicará una codificación por color según la tipología del alojamiento (*es_solo_habitacion*). Se incluirán tooltips para visualizar el título descriptivo del anuncio al posicionar el cursor.
- Objetivo: Evaluar la correlación espacial para determinar si existe una prima de precio demostrable por la cercanía al kilómetro cero turístico (zocodover/catedral), o si factores como la capacidad distorsionan esta regla.

C. Impacto Cualitativo: La "Prima" de los Servicios de Valor Añadido

- Tipo de gráfico: Gráficos de caja y bigotes (Boxplots paralelos) o Gráfico de Violín (Violin plot).
- Variables implicadas: Variables booleanas extraídas ("*tiene_piscina*", "*tiene_parking*", "*es_superanfitrión*") vs "*price*" (Eje Y).
- Objetivo: Analizar no solo el precio medio, sino la distribución estadística de los precios, para comprender cuánto valor real en euros (€) añade al alojamiento disponer de piscina, parking privado o gozar del estatus de "*Superanfitrión*" dentro de la plataforma.

Estos cuadros se consideran individuales (por ciudad) y por lo tanto se replicarán para cada uno de los destinos, permitiendo entender la dinámica local de cada ciudad de forma independiente.

En segundo lugar, y como aportación de máximo valor analítico al proyecto, se desarrollará un panel de Análisis Comparativo Interurbano utilizando herramientas visuales avanzadas en Altair para responder a las siguientes cuestiones:

1. Distribución y Volatilidad del Mercado (Gráfico de Cajas Facetado):

- Diseño: Un gráfico de cajas (Boxplot) enrejado o facetado mediante la función `.facet()` de Altair por cada ciudad, mostrando la distribución de precios en el eje Y bajo una misma escala compartida.
- Objetivo Analítico: Comparar visualmente qué ciudad tiene el coste base más alto y cuál presenta mayor dispersión de precios (por ejemplo, comprobar si el mercado de San Sebastián es significativamente más restrictivo y volátil que el de Toledo).

2. Curvas de Estacionalidad y Antelación (Gráfico de Líneas Superpuestas):

- Diseño: Un gráfico temporal donde el eje X sea la fecha de check-in o los días de antelación, el eje Y sea el precio medio, y cada línea represente a una ciudad distinta (codificada por color).
- Objetivo Analítico: Contrastar el comportamiento de los anfitriones según la zona geográfica. Permitirá descubrir, por ejemplo, si los precios en destinos de costa (Málaga, Valencia) sufren picos de encarecimiento de "última hora" mucho más agresivos en ciertas fechas que los destinos de turismo cultural continuo (Toledo).

3. El Valor Geográfico de los Extras (Gráfico de Barras Agrupadas):

- Diseño: Un gráfico de barras agrupadas (Grouped Bar Chart usando xOffset en Altair) que muestre el incremento porcentual medio del precio que aporta una característica concreta (ej. *“tiene_piscina”* o *“tiene_terraza_balcon”*) segmentado visualmente por ciudad.
- Objetivo Analítico: Demostrar cómo el valor de una misma amenity fluctúa dependiendo de la necesidad climática o cultural del mercado. ¿Se paga la misma prima por tener piscina en Toledo que en San Sebastián? Este gráfico resolverá esa incógnita de un solo vistazo.

Finalmente, y como parte adicional a la entrega del proyecto, se incluirá una infografía recopilando los gráficos con resultados más significativos del estudio.